

HUBCONNECT

ÇOK KİRAÇILI AJANS İŞLETİM SİSTEMİ — MİMARİ ŞEMA SETİ

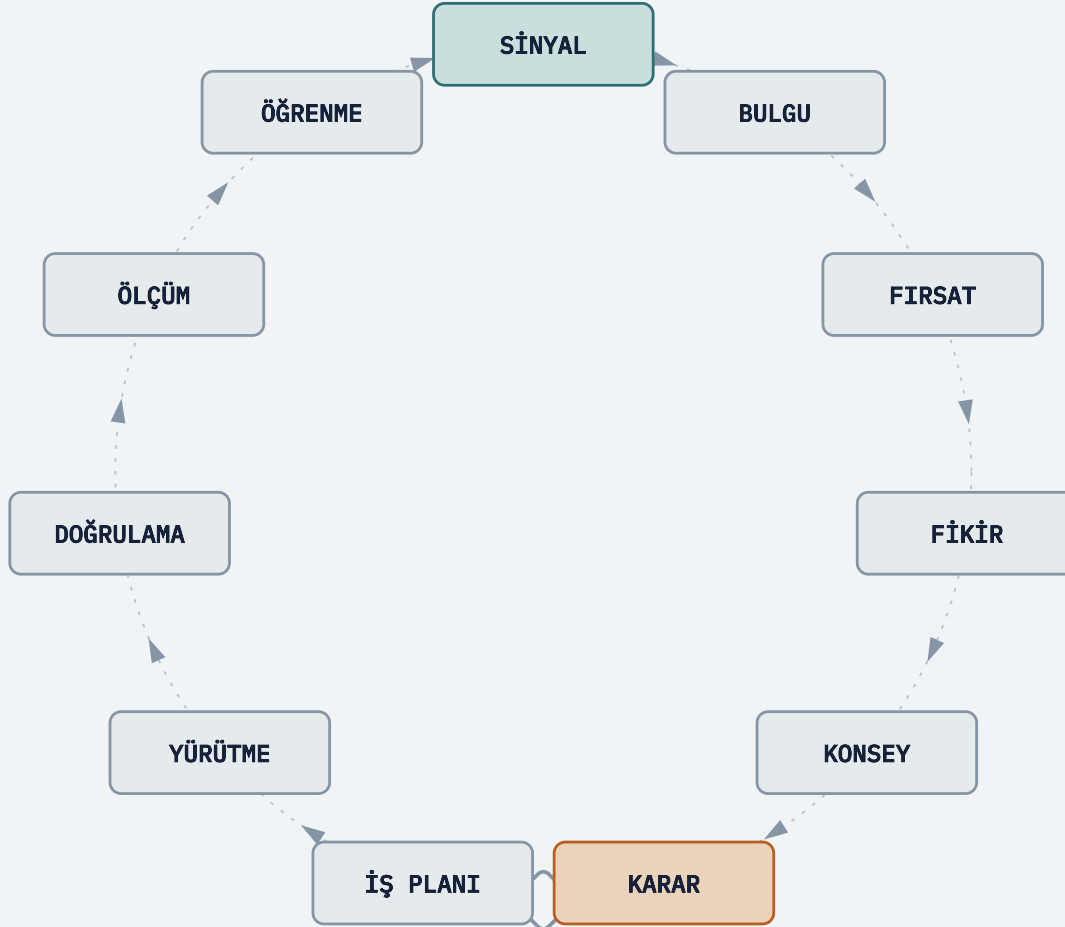
HubConnect, bir dijital ajansın birden çok marka için yürüttüğü araştırma, planlama, onay, yürütme, doğrulama ve öğrenme adımlarını tek çekirdekte toplayan bir "Agency OS" dur. Mikroservis değil, bilinçli tercih edilmiş modüler bir monolittir: iş mantığının büyük bölümü Next.js Server Actions üzerinden akar, arka planda PostgreSQL üzerine kurulu bir transactional-outbox kuyruğu çalışır ve otonom karar döngüsü riskli işlemlerde kendini durdurup insan onayı bekler.

DOKÜMAN NO HC-ARCH-001	REVIZYON A
TARİH 12.08.2026	MİMARİ Modüler Monolit
KIRACI MODELİ Workspace → Project → Brand	ÇEKİRDEK Next.js 16 · React 19
VERİ KATMANI PostgreSQL · Prisma 6	KAPSAM 66 model · 60+ enum

Otonom Ajans Döngüsü

ŞEKİL HC-01-A
11 AŞAMALI KAPALI DÖNGÜ

Sistem, dış dünyadan aldığı sinyalleri iş fırsatlarına, fikirlere ve ölçülebilir yürütmelere dönüştürüp sonuçlardan geri öğrenen kapalı bir döngü olarak çalışır. Her marka (Brand) için bu döngü sürekli döner; bir turun çıktısı bir sonrakinin girdisidir.



riske göre devreye girer · bkz. Sayfa 05

Halka boyunca akış saat yönündedir. **SİNYAL** düğümü dış dünyadan (rakip hareketleri, performans verisi, brief) gelen girdiyi; **KARAR** → **İŞ PLANI** geçişindeki kapı ise riskli işlemler için insan onayının devreye girdiği noktayı temsil eder.

AŞAMA	İLGİLİ VERİ MODELLERİ
Sinyal → Bulgu	Signal, Finding, Insight
Fırsat → Fikir	Opportunity, Idea
Konsey → Karar	CouncilEvaluation, AgencyDecision
İş Planı	WorkPlan, WorkHandoff
Yürütme → Doğrulama	Command, Task, ExecutionJob, ExecutionVerification (bkz. Sayfa 03)
Ölçüm → Öğrenme	MeasurementPlan, MeasurementCheck, BrandLearning

NEDEN DÖNGÜ?

Agency OS, tek seferlik bir betik değil sürekli çalışan bir sistemdir. Her tur bir markanın durumunu günceller ve ölçüm sonuçları bir sonraki sinyal değerlendirmesini besler.

TETİKLEYİCİLER

[AgencyTrigger](#) ve [AutonomyPolicy](#) modelleri, döngünün hangi koşullarda otomatik ilerleyeceğini, hangi noktalarda duracağını tanımlar.

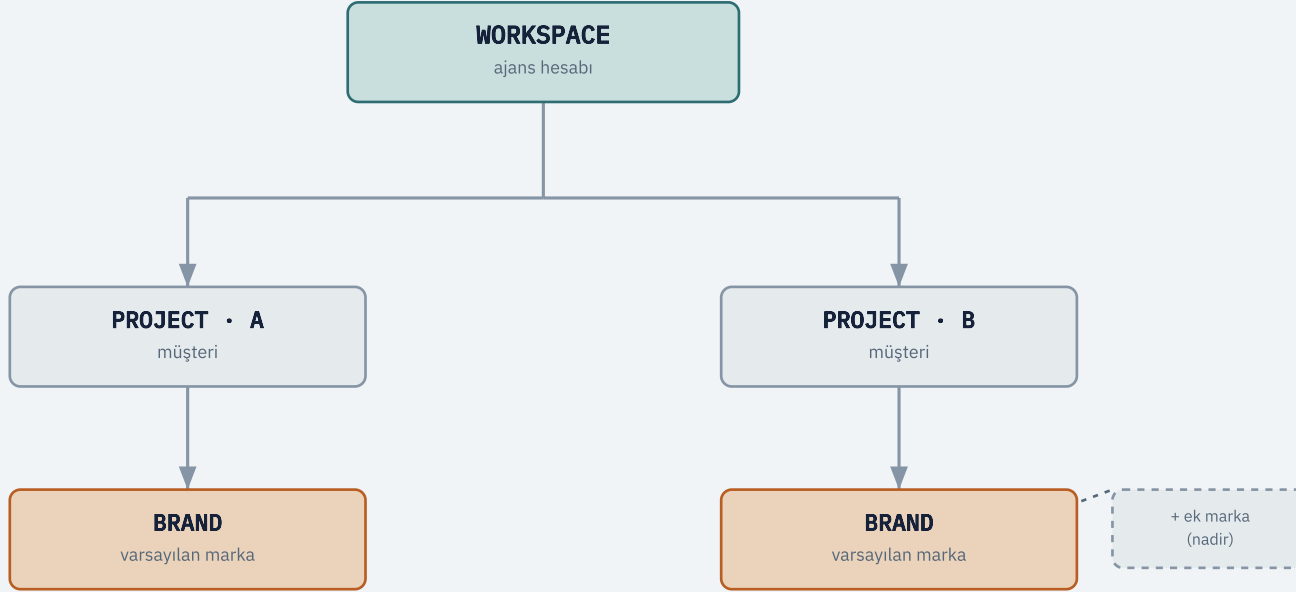
KAPSAM

Döngü marka (Brand) bazlıdır — bkz. [Sayfa 02](#), kiracı hiyerarşisi.

Kiracı Hiyerarşisi

ŞEKİL HC-02-A
WORKSPACE → PROJECT → BRAND

Çok kiracılılık üç kademelidir. Bir ajans hesabı (Workspace) birden çok müşteri (Project) barındırır; her proje genellikle tek bir varsayılan markaya (Brand) sahiptir, ancak şema birden fazlasına izin verir.



Her seviyede `workspaceId` / `projectId` / `brandId` alt tablolara bilinçli olarak tekrar yazılır (denormalizasyon) – böylece erişim kontrolü join gerektirmeden, tek satır üzerinden doğrulanabilir (bkz. Sayfa 04).

ÜYELİK ROLLERİ

OWNER

ADMIN

MEMBER

`WorkspaceMember` kaydı, bir kullanıcının hangi workspace'e hangi rolle bağlı olduğunu tutar.

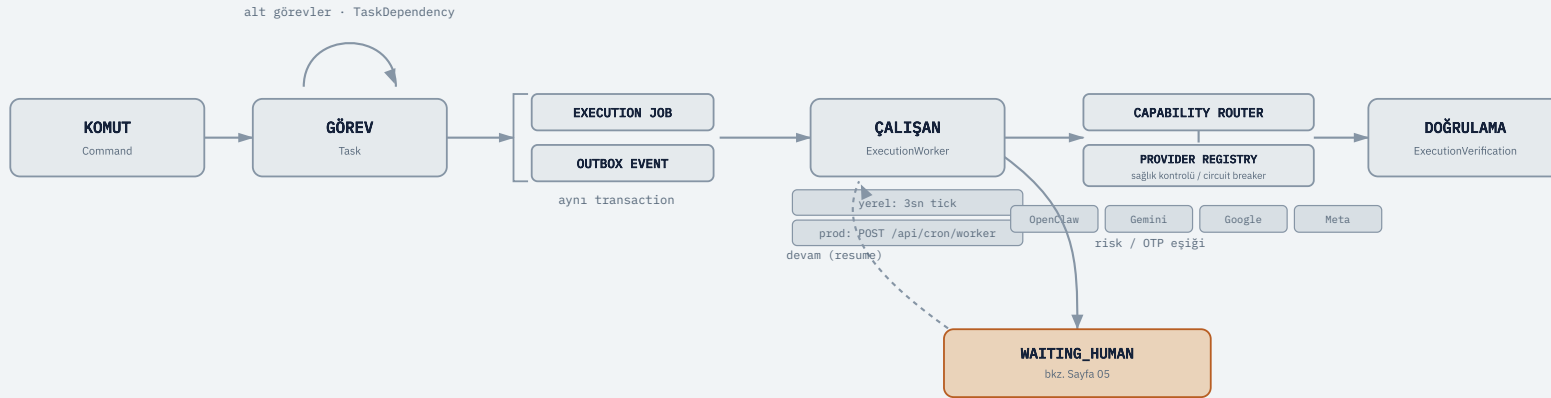
NEDEN TEKRAR (DENORMALIZE) ID?

Repository sorguları, bir kaydın doğru kiracıya ait olduğunu join zinciri kurmadan tek satırdan doğrulayabilir. Performans ve izolasyon garantisi bir arada.

Komut → Yürütme Hattı

ŞEKİL HC-03-A
TRANSACTIONAL OUTBOX DESENİ

Bir komutun kaynak eylemlere dönüşme hattı. ExecutionJob ile OutboxEvent aynı Prisma transaction'ında yazılır; kuyruk için Redis değil, doğrudan PostgreSQL satırları üzerinde polling kullanılır.



Sağlayıcı seçimi doğrudan değil **CapabilityKey** üzerinden yapılır; **ProviderRegistry** sağlık durumuna göre uygun sağlayıcıyı devreye sokar (circuit breaker).

NEDEN OUTBOX?

İş kaydı ile olay kaydı aynı transaction'da yazıldığı için "iş oluştu ama olay kayboldu" senaryosu yapısal olarak imkansızdır.

KUYRUK MEKANİZMASI

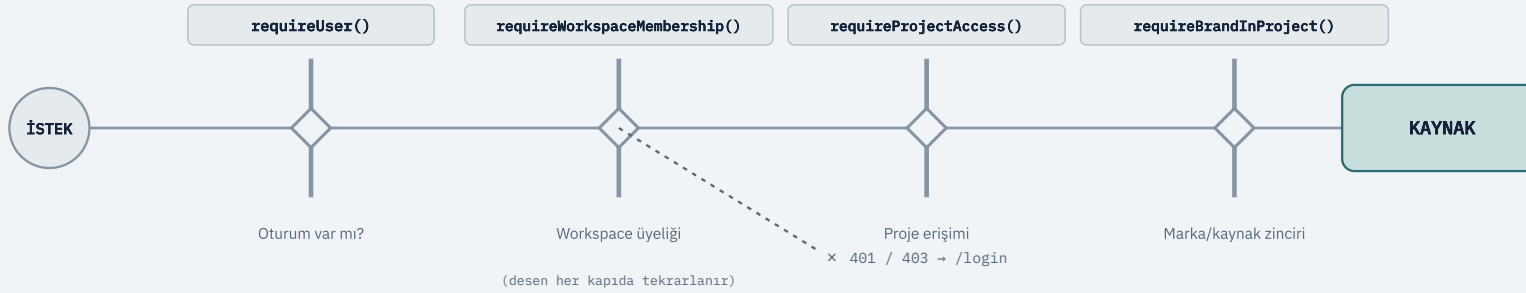
Redis/Docker yok — worker, `OutboxEvent` satırlarını doğrudan PostgreSQL üzerinden polling ile claim eder.

HATA DURUMLARI

`DeadLetterJob`, tekrarlanan başarısız denemeleri insan incelemesine ayırır.

Güvenlik Sınırları

Ayrı bir middleware .ts yoktur; her sayfa ve server action, isteği `src/server/security/tenant-context.ts` içindeki koruma fonksiyonlarından geçirir. Bir kapının doğrulayamadığı istek, kaynağa hiç ulaşmaz.



Örnek: bir müşterinin görev isteği başka bir müşterinin browser profiline yönlenebilir — `requireBrowserProfileInProject()` aynı desende bu IDOR sınıfı hatayı engeller.

KİMLİK DOĞRULAMA

Auth.js v5 · Credentials provider · bcrypt · JWT session · PrismaAdapter

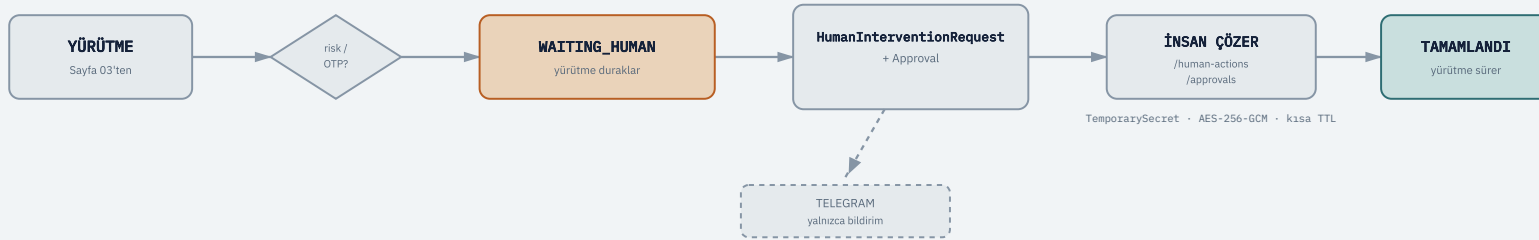
BOZUK OTURUM

`requireWorkspaceMembership()`, tutarsız bir session'ı doğrudan `/login`'e yönlendirir.

İnsan Müdahalesi Katmanı

ŞEKİL HC-05-A
WAITING_HUMAN DURUMU

Yürütme, OTP/MFA gibi bir risk eşiğine takıldığında kendini durdurur; iş bir insan gözden geçirip onaylayana kadar WAITING_HUMAN durumunda bekler.



Telegram, README'ye göre artık bir komut kaynağı değil — yalnızca dead-letter ve insan müdahalesi bildirimleri için tek yönlü bir kanaldır.

NEDEN AYRI BİR KISA ÖMÜRLÜ SIR?

`TemporarySecret`, OTP gibi tek kullanımlık verileri kısa TTL ile şifreli tutar; ana kimlik bilgisi deposuyla karışmaz.

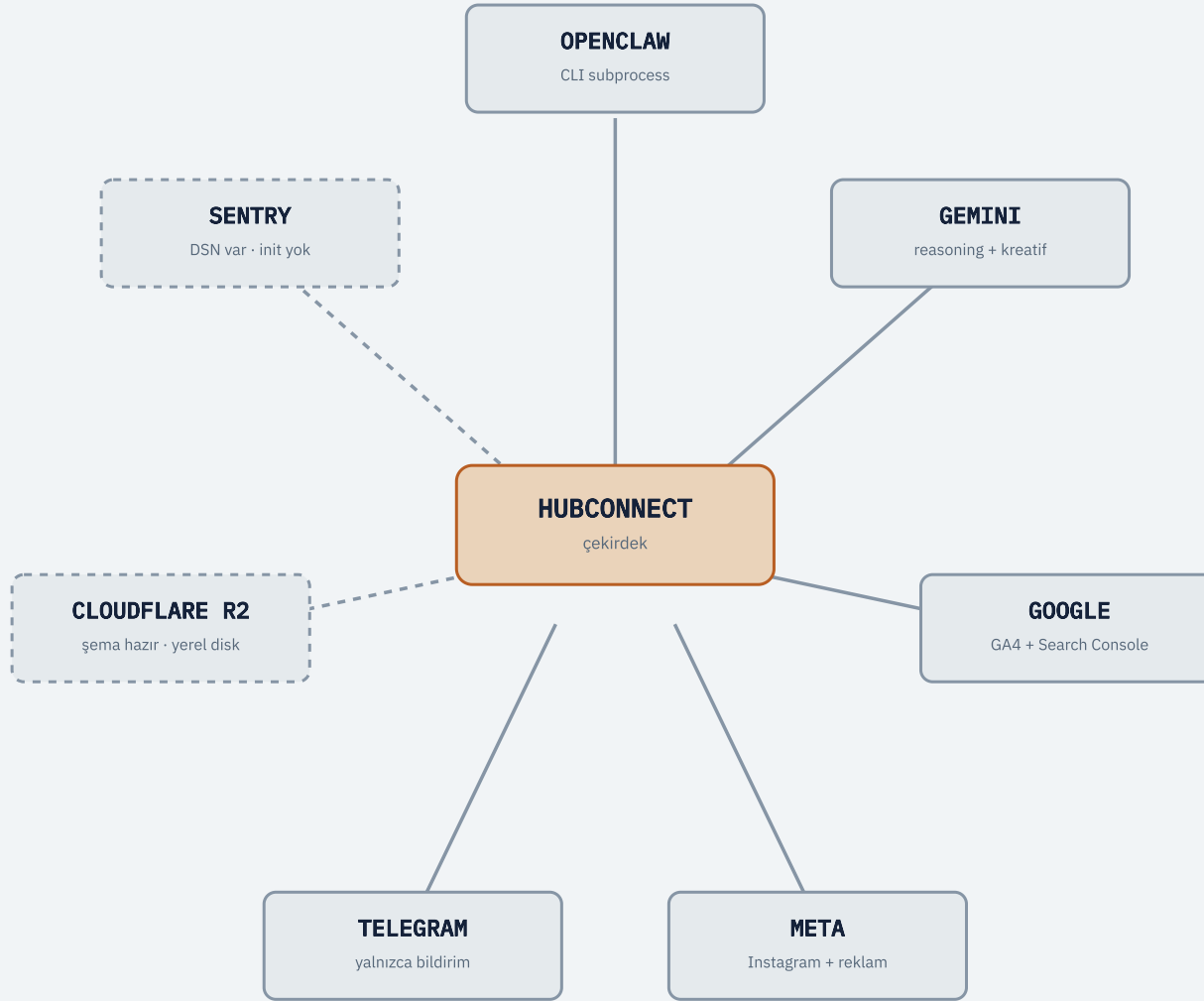
GÖZLEMLENEBİLİRLİK

`ProviderIncident` ve `ProviderHealth`, hangi sağlayıcının hangi sıklıkla insana düştüğünü izler.

Entegrasyon Haritası

ŞEKİL HC-06-A
KESİKLİ ÇİZGİ = HENÜZ BAĞLANMADI

Çekirdek, dış dünyayla yedi kanaldan konuşur. Kesikli çizgiler, şemada yeri ayrılmış ama henüz devreye alınmamış entegrasyonları gösterir.



OpenClaw, HTTP değil doğrudan `openclaw agent --json` alt sürecidir (subprocess);
Google ve Meta erişimi OAuth üzerinden, salt-okunur / yayın-kapsamlı olarak
sınırlandırılmıştır.

— aktif entegrasyon - - - şemada var, adapter henüz yazılmadı

GÖRSEL ÜRETİM

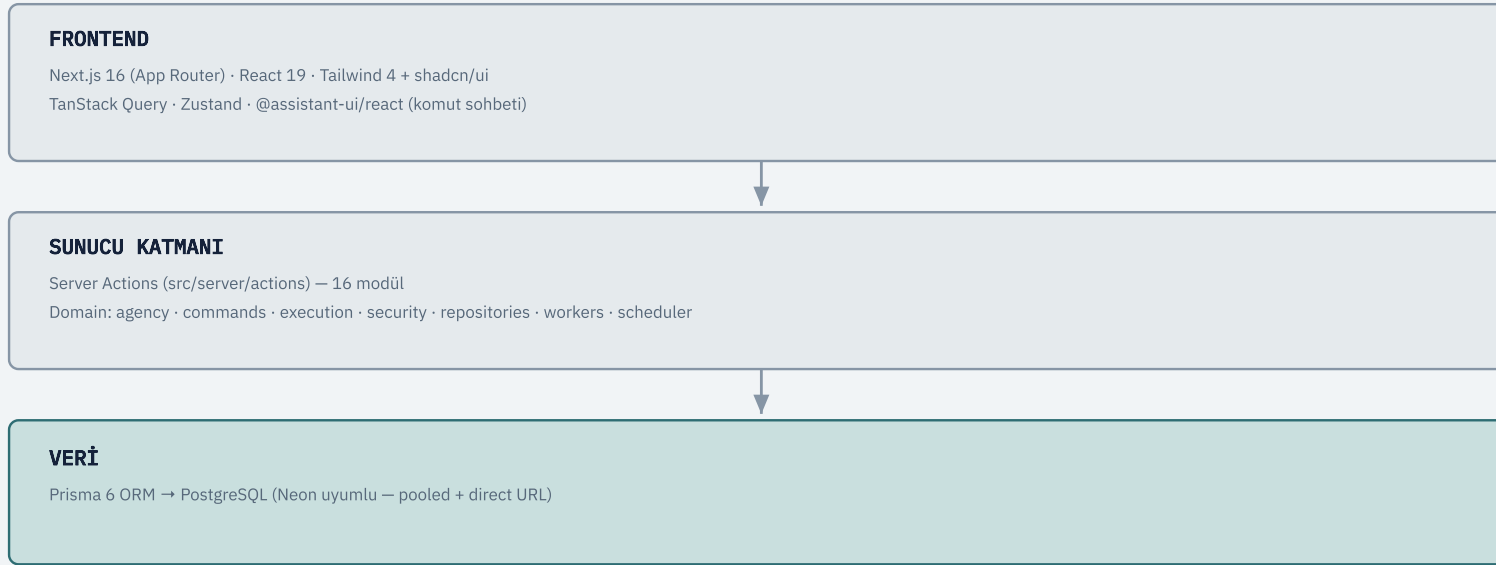
`openclaw infer image generate` — aynı CLI köprüsü üzerinden.

DEPOLAMA

Asset'ler şu an `storage/assets` altında yerel diskte tutuluyor; R2 adaptörü henüz devrede değil.

Yığın & Dağıtım

Sunucusuz (serverless) uyumsuz sürekli bir worker yerine, harici bir zamanlayıcının tetiklediği cron-tick modeli kullanılır.



66 model / 60'tan fazla enum, tek PostgreSQL şeması altında tutulur.



GitHub Actions ([.github/workflows/ci.yml](#)) — her adım bir öncekinin geçmesini bekler; geçici bir PostgreSQL servisi migration adımı için ayağa kaldırılır.

GELİŞTİRME MODU

`src/instrumentation.ts` içinde 3 saniyelik yerel tick çalışır;
`DISABLE_LOCAL_WORKER=true` ile kapatılabilir.

PRODÜKSİYON MODU

Harici zamanlayıcı, POST
`/api/cron/worker` uç noktasını
`Authorization: Bearer $CRON_SECRET` ile
tetikler — Railway benzeri uzun ömürlü bir
sunucu hedeflenir.

TEST

Vitest 2 — kritik yollar için birim ve
entegrasyon testleri.

ŞEMA SETİ DİZİNİ

01

Döngü

02

Kiracılık

03

Yürütme Hattı

04

Güvenlik

05

İnsan Müdahalesi

06

Entegrasyonlar

07

Yığın & Dağıtım

↑

Kapağa dön

Bu şema, HubConnect deposunun src/server, prisma/schema.prisma, src/lib ve README.md kaynak taramasına dayanılarak hazırlanmıştır · HC-ARCH-001
REV A · 12.08.2026